一、系统测试相关：

1. 随着用户对于 XX 搜索结果的个性化需求越来越多，如 XX、XX 等，详细的功能测试变得越来越重要，笔者将 XX 搜索功能拆解为核心搜索、过滤与排序、结果展示、异常处理四大功能模块。针对核心搜索，采用场景法与等价类划分，正向场景如 XX、XX 等。笔者设计了包括 XX 在内的 120 个基础用例。特别注重边界验证，如日期选择上，对当天、一年后极限日期及闰年 2 月 29 日的处理。通过 Selenium 和 Postman 构造和发送请求，验证返回的 XX 信息是否与功能需求一致。过滤与排序是提升用户体验的关键，笔者所在团队测试了 XX、XX 等多个维度的过滤与联动过滤功能。例如，XX。针对异常处理，采用错误推测方法，模拟各种异常输入，如 XX、XX、特殊字符注入等。系统均能返回清晰友好的错误提示，未发生崩溃或数据泄漏。

2. XX 搜索是典型的高并发、低延迟场景，需要 7*24 小时提供服务，QPS 可达 8000+，详尽的性能测试能够评估系统在特定条件下的运行表现，关注系统的非功能性需求，验证系统是否满足性能需求规格说明书中定义的指标。笔者收集所有用户非功能需求后，确定性能目标是：核心搜索接口在平均每秒 3000 次并发请求下，平均响应时间低于 600 毫秒，成功率 99.99%。对比性能测试工具 JMeter、LoadRunner、Apache Bench，Jmeter 具备开源免费、支持图形化界面、功能全面等优势，所以笔者所在团队选用 Jmeter 作为性能测试工具，并分阶段实施性能测试。基准测试阶段，模拟单用户最典型操作链（搜索-筛选-排序），确立性能基线。负载测试阶段，通过分析历史日志，模拟“节假日、大促搜索高峰”和“工作日出行”两种负载模型。节假日、大促模型突发性强，需在短时间内将并发用户从 2000 陡增至 5000，测试系统弹性。压力测试阶段，持续增加负载至每秒 8000 请求，观察系统在 CPU 使用率超过 85%、内存使用率达 90%时的表现，发现内存在极端压力下回收不及时的问题，通过优化线程重启机制解决该问题。稳定性测试则模拟 3000 用户并发持续运行 8 小时，监控内存泄漏和响应时间曲线，确保系统长时间运行平稳。针对复杂的 XX 查询，我们进行了专项性能剖析。利用 APM 工具 SkyWalking 定位到此类查询引发数据库复杂联合查询，耗时占整个请求的 70%。通过引入 Redis 缓存热门 XX 搜索结果，并对复杂查询结果进行短期缓存，将该类请求的平均响应时间从 2.5 秒优化至 900 毫秒。全链路全场景压测数据表明，系统在 5000 并发下，平均响应时间为 720 毫秒，满足预定目标。

3. 随着信创改造的不断深化，无论是采用混合云，还是采购物理机部署，都会面临单一信创芯片厂商生产力不足，无法满足全部 XX 搜索硬件数量需求的问题，所以 XX 搜索系统可以同时运行在 x86 和 ARM 架构的信创 CPU 上是十分必要的。这就使得兼容性测试十分重要，通过详细的兼容性测试消除因环境差异导致的用户体验割裂或功能失效。笔者所在团队设定的兼容性目标是系统在 x86 与 ARM 架构服务器上，所有核心业务功能均能正常运行，且性能差异不超过 10%。为了成本更优，且信创芯片种类丰富，我们选择了对信创芯片支持最好的天翼云，同时采用按需付费模型创建云服务器实例。针对 XX 搜索的核心服务，笔者分别在 x86 海光、鲲鹏芯片与 ARM

飞腾芯片的独立天翼云环境中部署，使用相同的测试数据集执行 API 接口测试，并关注功能一致性：相同查询条件返回的航班数量、价格、排序结果必须完全一致。性能基准对比：使用同一套 Jmeter 性能测试脚本，在同等资源配置的 x86 与 ARM 服务器上运行，记录关键指标。全链路兼容性测试结果表明，系统在 x86 架构（海光 7390、鲲鹏 920 CPU）与 ARM 架构（飞腾 5000C）集群中功能完全一致，核心搜索接口在 3000 并发下，x86 架构平均响应时间为 580 毫秒，ARM 架构为 620 毫秒，差异在允许范围内。

4. XX 系统采用混合云部署架构，同时部署在天翼云和我司自建 IDC，不过不论部在多少个机房中，网络分区、宿主机故障都是有可能发生的，所以笔者采用故障注入测试验证 XX 搜索系统在局部故障发生时，是否具备故障隔离、熔断、重试、主备切换等容错能力，确保局部故障不影响系统整体服务的正常运行，核心业务不间断。采用 Chaos Mesh 工具 Kill 掉 XX 模块的一个应用实例，模拟微服务单节点故障。测试前通过 JMeter 持续模拟用户 XX 查询请求，保证流量稳定。故障注入后，负载均衡健康检查在 2 秒内自动摘除故障节点，并将用户请求转发至其余正常节点，服务成功率保持 100%，用户无任何感知。同时，Kubernetes 集群在 8 秒内自动重建实例，完成服务自愈。监控系统及时产生异常告警，实现故障快速发现与定位。测试结果表明，系统的服务发现、负载均衡、故障自愈机制均达到设计目标，能够有效抵御微服务单节点故障风险，保障核心业务连续稳定运行，满足高可靠性架构要求。

5. 虽然用户在客户端会对查询请求参数进行校验，但在版本更新后可能会产生请求参数填写错误情况，所以容错测试主要针对接口参数校验进行。采用 JMeter 工具搭建测试脚本，注入异常请求，批量构造包含非法参数的查询请求，模拟真实用户误操作，经过测试系统能够快速完成参数校验，对包含特殊字符、超出范围日期、不存在航班号的非法请求，立即拦截并返回友好的中文提示（“请输入正确的航班号。查询日期需在近 30 天内，请重新选择”），不返回 500、404 等错误页。

二、云计算-服务网格相关：

1. 在 XX 搜索系统项目中，由于用户的个性化需求越来越多，如 XX、XX 等，导致微服务的数量也是成倍增加，这使得微服务间通信、流量管理成为了开发过程中的一大难点，结合 XX 搜索系统的微服务架构现状，笔者所在团队决定采用“Istio+Kubernetes”的部署模式，构建服务网格的控制面与数据平面，实现对所有微服务的统一治理，架构设计遵循“业务无侵入、治理标准化、扩展可灵活”的原则。整体架构分为三层：业务服务层、服务网格层、基础设施层。业务服务层：包含 XX 搜索系统的所有核心微服务：搜索接入服务、缓存服务、价格计算服务、日志收集服务等，所有微服务均部署于 Kubernetes 集群，采用容器化部署，支持弹性伸缩，为服务网格层提供治理对象。服务网格层：核心采用 Istio 服务网格，分为控制面与数据平面，实现对业务服务的无侵入式治理。数据平面：由 Envoy 代理组成，采用 Sidecar 模式，承担流量转发、监控数据采集、故障治理等核心职责。控

制面：采用 Istio 的 Istiod 组件，负责统一配置管理、服务发现确保治理规则的一致性与标准化。基础设施层：包含 Kubernetes 集群、监控告警系统（Prometheus + Grafana）、日志分析系统（Filebeat + Logstash + Elasticsearch + Kibana）。

2. XX 搜索系统有着搜索系统典型的波峰波谷差异极大的特点，为避免流量高峰导致系统响应时间慢，同时缓解流量低估 CPU 使用率低，资源冗余，运行成本过高问题。针对节假日、促销等场景的流量峰值，笔者所在团队选择基于 Istio 的 VirtualService 与 DestinationRule 资源，配置精细化的限流规则，重点保护核心计算服务，避免核心服务过载。我们根据历史流量数据，为每个服务设定差异化的限流阈值:搜索接入服务限流阈值为 10000 QPS,航班数据服务限流阈值为 25000 QPS，价格计算服务限流阈值为 5000 QPS；同时配置降级机制，对于超出限流阈值的请求，返回友好提示“当前搜索人数过多，请稍后再试”，而非直接拒绝。此外，结合天翼云弹性伸缩组与 Kubernetes 的弹性伸缩能力，当集群整体的 CPU、内存使用率达到阈值时，自动扩容服务实例，配合服务网格的限流规则，实现“限流+扩容”双重保障，确保峰值时段系统的稳定性。在“双十一”、春节等峰值时段，系统未出现核心服务过载、雪崩等问题。

3. 针对 XX 搜索系统服务多，且依赖复杂、故障排查低效的痛点，基于 Istio 的可观测性功能，结合监控报警、日志工具，构建全链路可观测体系，实现“监控可视化、故障可追溯”，基于 Istio 的 Prometheus 监控插件，采集服务网格与业务服务的核心运行指标，包括：服务调用指标（QPS、响应延迟、错误率）、Envoy 代理指标（流量转发量、连接数、缓存命中率）、服务实例指标（CPU、内存使用率），采集数据到 Grafana 并可视化展示。同时配置天翼云 Alarm Agent 告警规则，当指标超出预设阈值时，通过短信、电话、邮件等方式及时通知运维人员，实现故障的提前预判与主动响应，系统潜在故障预判准确率持续提高。为满足解决用户提出的问题工单，可能会用到较久之前的日志排查，故系统的运行日志、服务日志都需要收集。整合 Istio 的 Envoy 日志与业务服务日志，Envoy 代理自动收集所有服务间调用的请求头、响应头、调用时长、错误等信息，业务服务日志通过 Filebeat 采集，通过 Logstash 上传至 Elasticsearch，通过 Kibana 可视化展示，支持按服务名称、时间范围等维度检索日志。系统运行稳定性进一步提升，构建了“日志-监控-追踪”三位一体的可观测体系，为系统架构优化提供了数据支撑。

三、云计算-论存储计算分离

1. 传统搜索系统多采用存算一体架构，计算节点与存储节点耦合部署，即每台服务器同时承担数据存储与查询计算功能。这种架构在系统规模较小时能够满足基本需求，但随着业务复杂度提升，其局限性日益凸显：一是资源利用率低，计算高峰期 CPU、内存满负载而存储资源闲置；二是扩展性受限，计算与存储需同步扩容，无法根据业务需求独立调整，扩容成本高、周期长；三是性能瓶颈明显，单一节点难以支撑高并发查询与实时数据更新；四是运维复杂度高，节点故障可能同时影响计

算任务与数据可用性，故障排查与恢复难度大。结合 XX 搜索系统的业务需求，存储计算分离技术主要具备以下四大关键特性，能够精准匹配系统的核心诉求：一是资源弹性扩展，计算层与存储层独立部署，可根据业务需求分别进行扩容或缩容。二是高可用与高可靠，计算层采用集群部署，单个计算节点故障时，其他节点可快速接管其计算任务，不影响系统的整体服务。三是运维成本降低，计算层与存储层的职责分离，运维人员可分别针对计算集群与存储集群进行维护、监控和故障排查，无需关注两者之间的耦合关系。

2. 存储层的设计重点是实现数据的分层存储、高可用存储与快速检索、降低存储成本，结合 XX 搜索实时性数据、高频访问数据、海量历史数据特点，将存储层分为三个独立的存储集群，分别负责不同类型数据的存储与管理。业务数据存储集群：相比于传统的 MySQL 等数据库，TiDB 具备分布式事务支持、高并发访问、低延迟读写等特性，所以笔者采用分布式数据库（TiDB）部署，负责存储航班信息、实时票价、舱位库存、用户查询记录等实时性高、访问频率高的业务数据。缓存存储集群：采用分布式缓存（Redis）部署，负责缓存复杂的、高频查询结果、热点数据，作为计算层与存储层之间的缓冲，减少计算层对存储层的访问压力，提升查询响应速度。缓存集群采用主从复制+哨兵模式，实现高可用。历史数据存储集群：由于用户在退改时需要查询三年内的票价数据，同时访问量不大，对于这种“温数据”、“冷数据”，我们采用分布式文件系统（HDFS）部署，负责存储历史数据、归档数据、用户查询日志等海量历史数据，这些数据实时性要求低，但数据量庞大，需长期存储，对一年前的数据采用 SAS 盘存储，两年到三年前的数据采用 SATA 盘存储，兼顾性能与成本。相比于传统的存储计算耦合架构，通过冷热数据分层与 TiDB 高压缩特性，在保障高可用、高性能的同时存储硬件投入降低 30%左右。

3. 计算层作为系统的核心计算模块，重点是提升并发处理能力、降低计算延迟，同时实现资源的弹性调度。结合 XX 搜索系统的查询场景，计算层采用“分片+集群”的部署模式。计算节点分片：根据用户查询的请求对计算节点进行分片，每个分片负责处理特定的查询请求，避免不同的查询请求相互干扰，提升计算效率。由于分片后服务过多，难以治理同时，笔者引入 istio 服务网格技术，请求通过网关后自动路由到对应服务并访问到对应数据存储层服务，降低服务治理难度。

4. XX 搜索系统具有高并发且流量波动特性，并发量高峰与低估相差 5 倍由于，为解决流量高峰响应延迟、流量低估资源使用率的，资源浪费问题。引入弹性伸缩机制，计算与存储分离架构能够很好的支持这种弹性伸缩场景，计算节点不保存海量数据，弹性伸缩用时极短，结合天翼云弹性伸缩组可以在秒级拉起计算节点实例，并基于 Kubernetes 自动扩缩容功能，设置阈值，当集群 CPU 利用率超过 75%，系统自动扩容计算节点，提升并发处理能力；高峰期过后，自动缩容节点，释放资源。解决了平日流量低谷期资源利用率低、硬件资源冗余，查询高峰资源使用率超限，导致长时间限流问题。

可靠性设计相关：

1. 对比单体架构任意模块故障可能导致整个应用不可用，哪怕是辅助功能日志记录模块写盘失败都会导致整个应用不可用，这是不满足 XX 搜索系统可靠性需求的，同时单体架构难以支持灰度更新，再小的功能修改都需要全量更新代码，这就需要更多的资源冗余，每次版本上线都是对系统可靠性的考验，为解决上述问题，保障 XX 搜索系统可以 7*24 小时为旅客提供服务，笔者所在团队按照“单一职责”原则，将原有单体系统拆分为，如 XX 搜索服务、航班动态服务、缓存服务、日志服务等多个独立的微服务，每个服务专注于一项核心业务，服务之间通过轻量级 HTTP 接口通信，实现模块解耦。每个微服务按业务独立部署运行，单个服务异常仅影响局部功能，不会引发系统级雪崩，大幅提升整体可靠性，同时可对单个服务进行迭代、回滚、灰度发布，变更范围可控，显著降低发布导致的整体不可用概率。

2. 2024 年 4 月腾讯云控制台 API 异常，同年 8 月阿里云上海可用区 N 发生网络故障无法访问，为保障系统可靠性，笔者决定采用混合云架构，将核心微服务部署在天翼云华北和华东 ECS 实例上，同时在我司北京本地 IDC 保留备份节点，通过专线实现云上云下环境数据同步，避免单一机房故障导致系统中断。同时使用智能 DNS 和天翼云原生负载均衡，智能 DNS 保障如果某一机房出现大面服务无法访问直接隔离其负载均衡避免大面积报错。负载均衡采用最小连接策略以解决不同请求复杂程度不同导致负载不均的问题，同时配置健康检查，每两秒向 XX 搜索服务的 5001 端口发送健康检查请求，如果连续三次返回非 2xx 则自动剔除该实例，有效避免由硬件故障或应用故障导致的应用持续无法提供服务问题。

3. 由于一个用户的查询请求可能对应多个服务内请求，所以只根据用户请求 QPS 指标和集群整体 CPU 使用率来进行弹性伸缩是不够的。所以系统仍需容错机制，笔者引入微服务高可用流量管理 Sentinel 框架，实现服务熔断、降级功能，避免故障扩散。熔断针对航班舱位动态服务，设置熔断阈值，10 秒内请求错误率超过 50%，当达到阈值时，触发熔断，暂时停止调用该服务，避免重试风暴导致服务进一步恶化，熔断时长设置为 30 秒，之后自动尝试恢复调用。降级机制，根据业务优先级，将系统功能分为核心的 XX 搜索服务与非核心的历史查询功能，当系统负载过高或出现故障时，自动降级非核心功能，高峰时段降级历史查询功能，返回“当前查询高峰，暂不支持历史查询”提示，确保 XX 搜索功能的响应速度。

4. 为了进一步提高系统可靠性，笔者在系统运维方面也采用了可靠性设计，全链路监控，实现系统运行状态的全面可视化。日志监控：采用 ELK 日志收集与分析系统，收集所有服务的日志，实现日志的集中存储、检索与分析，快速定位故障原因。指标监控：采用 Prometheus+Grafana，监控系统核心指标，包括服务实例的 CPU、内存、磁盘利用率、响应时间，并设置指标阈值，触发告警。结合天翼云原生 Alarm Agent 建立分级告警机制，根据故障严重程度，将告警分为紧急告警（如系统崩溃、核心服务不可用）、重要告警（如接口错误率升高、缓存命中率下降）、普通告警

（如非核心服务负载过高），不同级别告警采用不同的通知方式（紧急告警：电话+短信+邮件，重要告警：短信+邮件，普通告警：邮件），确保运维人员能够及时接收告警信息。

架构演化相关：

1. XX 系统作为 XX 服务的核心入口，需保障 7×24 小时稳定运行，在架构演化过程中必须遵循渐进式原则。分阶段、分模块推进微服务拆分、信创替代与混合云部署，确保新旧架构平滑衔接，降低演化风险。具体实施分为两个阶段，第一阶段为试点验证阶段，聚焦核心非关键业务的微服务部署在信创服务器试点。选择 QPS 低且有个性化需求的用户，如 XX 查询功能，拆分成为独立的微服务，并且部署在天翼云信创 ECS 实例上，进行试点运行。此阶段核心目标是验证技术方案的可行性，降低后续大规模演化的风险。第二阶段，全面推广阶段，逐步推进核心业务微服务信创替代与混合云全面部署。在试点验证通过的基础上，按照业务优先级，逐步将 XX、XX 等核心业务拆分为独立微服务，逐步迁移至天翼云信创环境。在迁移的过程中同时保留我司 IDC 非信创环境三个月作为备份，如果信创环境出现难以预估的故障，直接通过接口层服务切换流量回我司 IDC 非信创环境，确保业务不中断。

2. 在信创替代的并行期两套环境并行运行，资源冗余，势必会增加系统运行成本，为满足我司降本增效的整体原则，笔者所在团队遵循成本可控原则，尽可能减少并行期成本。在信创 CPU 兼容性测试阶段，需开通大量天翼云信创 ECS 实例，但由于测试工作基本只在工作时间开展，所以造成测试资源使用率并不高。所以笔者将包年包月的天翼云实例转为按量付费，同时在代码更新后自动构建系统镜像，结合天翼云弹性伸缩服务，配置定期弹性伸缩策略，在非工作时间释放 CPU 使用率低于 20%的测试环境 ECS 实例，在工作时间前 30 分钟通过系统镜像创建 ECS 实例，同时配置 ECS 启动项，在创建时执行服务注册，数据同步等脚本，完成服务准备。与包年包月付费模式相比，这种按量付费配合弹性伸缩的模式，将测试环境的 ECS 云服务器成本减少了 40%，成功控制了并行期系统成本。

基于架构的软件设计相关：

1. 首先，业务需求梳理。XX 搜索系统的核心业务是接收用户的搜索请求，聚合多数据源的 XX、XX，返回符合用户需求的搜索结果，并支持用户进一步的 XX 操作。结合 XX 平台的业务特点，系统还需支持 XX、XX 等增值业务需求，同时需对接平台的 XX 系统、XX 系统，实现业务流程的闭环。其次，质量属性需求明确，结合 XX 搜索系统的业务特点，我们将质量属性需求作为架构设计的核心约束，明确了以下关键目标：性能需求，日均查询请求量达 1 亿次以上，峰值并发量达 8000QPS，平均响应延迟≤1000ms；可用性需求，系统全年可用性≥99.99%，支持故障自动恢复，故障恢复时间≤5 分钟；可扩展性需求，支持业务功能快速迭代；可伸缩性，支持服务器集群横向扩展；安全性需求，支持请求限流、防爬虫攻击、系统数据安全。本阶段笔者所在团队通过多次需求评审，解决了需求模糊、需求冲突等问题，形成了《XX

搜索系统架构需求规格说明书》，为后续的架构设计提供了明确的依据，确保架构设计能够精准匹配业务需求和质量属性目标。

2. 架构风格选型，针对系统的核心需求，笔者对比了多种架构风格的优缺点：分层架构结构清晰、易于维护，但并发处理能力有限；微服务架构可扩展性强、便于迭代，但部署和运维复杂；管道-过滤器架构适合数据处理流程化场景，但灵活性不足。综合考虑后，我们采用“分层架构+微服务架构”的混合架构风格，既保留分层架构的清晰性和可维护性，又借助微服务架构的可扩展性和高并发处理能力，满足系统的质量属性需求。同时由于信创替代要求不断深化，笔者详细对比了阿里、华为、腾讯、天天翼公有云厂商对信创芯片的支持程度，最终选定对信创芯片支持程度最高的天翼云作为公有云厂商，并采用两地三中心的混合云物理架构，提高 XX 搜索系统的容灾能力，随提高了系统运行的成本，但对系统的可用性、业务连续性来说都有很大的提高。基于这种混合架构风格，我们将 XX 搜索系统的整体架构分为 4 层：数据源层负责整合各类数据源、基础支撑层为整个系统提供基础技术支撑、核心业务层负责实现各类业务逻辑、接口层负责接收用户的搜索请求，各层职责清晰、松耦合，便于开发、维护和扩展。本阶段完成了系统的整体架构设计和详细构件设计，明确了系统的架构风格、分层结构、构件职责、接口定义和质量属性保障策略，为后续的架构实现提供了清晰的指导。

3. 架构实现是 ABSD 方法的落地环节，开展构件开发、构件组装和系统集成工作。在本项目中，我们采用迭代式开发模式，分三个迭代周期完成架构实现工作，每个迭代周期聚焦于部分核心构件的开发和集成，确保开发质量和进度。首先，开发环境搭建与技术选型落地。基于架构设计文档中的技术选型，我们搭建了统一的开发、测试和部署环境，明确了开发规范和编码标准，确保开发工作的标准化。具体技术选型如下：后端开发采用 Java 语言，Spring Cloud 框架实现微服务架构；缓存采用 Redis 集群，实现多级缓存机制；数据库采用 NewSQL TiDB；负载均衡采用天翼云 ELB；日志监控采用 ELK 栈；部署采用 Docker 容器化和 Kubernetes 编排，实现系统的弹性扩展和自动化部署。同时，我们基于开源工具搭建了整套 DevOps 框架，提升开发、测试、部署的效率。其次，构件开发与编码实现。确保构件的高内聚、低耦合。例如，在缓存构件开发过程中，我们实现了缓存 key 生成规则、缓存存储、缓存查询、缓存失效、缓存预热等功能，支持 TTL 动态调整，通过遗传算法计算最优 TTL，核心场景缓存命中率提升至 47%。同时重点关注容错机制，在核心服务中增加重试、降级、熔断等容错机制，提升系统的可用性。

Devops 相关：

1. 第一是我们团队用 Jenkins 搭建持续集成流水线，开发提交代码后自动触发编译、单元测试和集成测试，4 小时内就能反馈问题。在某次开发修改主观题采分逻辑后，因为事先设置单元测试，所以 Jenkins 可以立即测出“XX”的 BUG，提前在测试阶段把此 BUG 段修复。并且在 Jenkins 流水线中我们配置了代码提交后自动编译，

某次添加了新功能用到了新的动态链接库，在自动编译阶段就发现了动态链接库缺失，避免了上线时基础环境缺失导致功能无法使用。第二部署则用 Ansible 编写标准化脚本，包含环境预检查、配置文件模板化替换、服务灰度更新等步骤，将测试环境到生产环境的部署时间从 1 天压缩至 30 分钟，大大降低了运维的时间成本。并且固化了上线步骤，解决了每次上线步骤不统一，手工执行命令容易出错的问题。其实 DevOps 的核心不是工具本身，而是通过流程自动化将“不可控的人工操作”转化为“可重复的标准步骤”，这才是保障 XX 搜索系统按时、高质量交付的关键。

2. DevOps 的跨团队协作机制是保障 XX 搜索系统稳定运行与需求响应能力的关键。在负责 XX 搜索系统迭代优化时，我曾深刻体会到团队协作脱节带来的痛点——系统需支撑每年至少 2 次的 XX 公司“大促”数据处理，且涉及 XX 等敏感信息，安全和稳定性要求极高，但最初开发、运维、测试团队各管一段：开发在需求分析阶段只关注功能实现，比如设计“高并发 XX 搜索模块”时，未与运维确认服务器 CPU 资源，导致测试环境用单卡 CPU 能跑通，但在实际生产环境因吞吐量过大，处理速度下降，响应时间增长。特别严重的是，会在“大促秒杀”时刻出现请求积压，被迫降级服务。这让我意识到，传统“开发做需求、运维管部署、测试找 BUG”的模式完全不适用于需高频迭代的 XX 搜索系统。于是我们组建开发、运维、测试联合工作组，推行全生命周期参与：需求分析阶段就拉通评审，比如针对“XX 需求”，三方共同明确数据压缩策略、压缩格式、数据同步频率等。这套机制落地后，效果显著：之前平均每次迭代发现 10 个问题，现在平均仅 5 个且均在需求或开发早期解决。其实跨团队协作的核心不是形式上的开会，而是让每个角色从“被动配合”转为“主动担责”，在需求源头就把稳定和安全的根基扎牢，这才是 XX 搜索系统高效运转的保障。